HACKWITHZACH · VIBE-CODING LAB · CHAPTER 14

Chapter 14 · The Widget Store

One request, two answers. Ask an assistant for the same store twice, once the way anyone asks and once with the control map attached. Nothing about the model changes. Everything about the result does.

21	14	0	6
GAPS IN THE VIBE-CODED STORE	OF THEM HIGH	GAPS IN THE HARDENED STORE	ATTACKS, SIDE BY SIDE

Two stores, one prompt | **Live attack bench** | **Real scanner, real counts** | **All on localhost**

The chapter where you stop trusting a finished-looking app. Cybersecurity education that gets you hired, promoted and paid · hackwithzach.com

© 2026 Vigilantia Technologies INC. · HackWithZach

THE LAB

One prompt, answered twice

This is the application example of the chapter: a finished-looking web app with a login, a catalog, a search box, reviews, an order history, a cart, a checkout, an admin console and a chatbot. It looks done. That is the point. `weak/app.py` is the vibe-coded answer, unedited. `hardened/app.py` is the same ask with the control map attached. Both read one schema from `seed.py`, so every attack on one has an exact counterpart on the other.

The weak store is evidence, not a draft to fix

Identity in the URL, string-interpolated SQL, plaintext passwords, a shell sink, a model reply parsed as a command, and admin mixed into the storefront. These are not caricatures. They are the defaults a model emits when the ask does not forbid them.

© 2026 Vigilantia Technologies INC. · HackWithZach

THE SIX ATTACKS

Break the weak store, watch the hardened one refuse

Attack	The vibe-coded store	The hardened store
Walk into admin	Role rides in the URL, admin in the same app: one changed word opens the admin page.	No admin route in the storefront. Admin is a separate service on 5202. 404.
Read another's order	Order 1003 belongs to someone else; the weak store hands it over.	Query matches id AND owner, and needs a session. Refused.
Set the price with a sentence	One chatbot message sends every price to zero.	The assistant has a read-only role and no price capability. Prices hold.
SQL in the search box	SQL built by string formatting.	The term is a bound parameter; the WAF refuses the pattern at the door with a 403.
Buy the catalog for nothing	With prices zeroed, fill the cart and check out at $0.00.	Total recomputed from list_price at checkout, a column only the admin service writes.
Find the API key	The weak store renders it into the page script.	The key is read server side and never reaches the browser.

Open `http://127.0.0.1:5115/attack`. The bench proxies the live store through its own origin, so the frame always renders. A toggle flips the same buttons between the weak store and the hardened one. Everything stays on localhost.

© 2026 Vigilantia Technologies INC. · HackWithZach

THE CONTROLS

What the hardened store does instead

Control	How it is shaped, and why
Admin isolation	A separate Flask service on 5202, its own login and cookie. A storefront compromise cannot reach a plane that is not in its process.
Pricing authority	list_price is written only by admin.py and read everywhere else. There is no code path from a customer, or the model, to a price.
Assistant least privilege	A named read-only role: catalog.read and answer.text allowed; pricing, orders, users, shell and secrets denied. The price-by-sentence bug is a privilege bug, not a prompt bug.
WAF, as defense in depth	waf.py screens every request for the OWASP input classes and for injection, before any view runs, and logs every hit. It is the layer in front, not the control.
Validation and encoding	guardrails.py types and bounds every request value at the edge and escapes every rendered value at the sink. Jinja autoescape, no |safe.
Structured audit	audit.py writes one JSON line per security event to file and stderr, with no network dependency. The admin console reads the tail, so the trail is in the product.

© 2026 Vigilantia Technologies INC. · HackWithZach

RUN IT YOURSELF

Every command, in order

No AWS, no cost: the whole lab runs on localhost. The scanner counts come from running appscan against each file, never from a number typed by hand.

```
cd module-07-vibe-coding/widget-store
pip install -r requirements.txt
python3 seed.py                          # builds both store databases
python3 appscan/appscan.py --selftest    # prove the gate before you trust it

# the four services
python3 weak/app.py                                          # 5200
WIDGET_LAB_DEV=1 WIDGET_LLM_API_KEY=sk-demo python3 hardened/app.py   # 5201
WIDGET_LAB_DEV=1 python3 hardened/admin.py                   # 5202
python3 visual/app.py                                        # 5115

# then open the teaching console and the live bench
#   http://127.0.0.1:5115        the console
#   http://127.0.0.1:5115/attack the six attacks, side by side

# prove the counts
python3 appscan/appscan.py weak/app.py       # NAKED. 21 gaps. 14 HIGH. exit 1
python3 appscan/appscan.py hardened/app.py   # PASS. No findings. exit 0
python3 appscan/appscan.py hardened/admin.py # PASS. No findings. exit 0
```

The bar got higher, the gaps did not get worse

This rebuild holds the store to five more controls than the first version: a WAF, input validation at the edge, admin-plane isolation, assistant least-privilege, and audit logging. So the weak store now reports twenty-one gaps where it used to report sixteen. That is the lesson, not a regression.

© 2026 Vigilantia Technologies INC. · HackWithZach

ABOUT

Who is behind this



Zach Marcy Cybersecurity Architect

Zach Marcy is a cybersecurity architect and mentor with over twenty years in IT and six in cybersecurity. He teaches cloud, AI and API security the way it is actually practised: every concept lands as something you build, break and then harden yourself. He teaches through HackWithZach on YouTube and Instagram and through the Cloud and AI Security Engineer program.

Jump in with the cohort

Connect	Where to find it
The Discord	hackwithzach.com/discord · questions between calls, wins, and whatever we are on this week
The program	academy.hackwithzach.com
Everything else	hackwithzach.com
YouTube / Instagram	@HackWithZach
This lab's code	github.com/hackwithzach-labs/hwz-cloud-foundation

DISCLAIMER

Read this before you deploy or run anything

Educational purpose only

These Materials are provided for education and training. They are not a production system, a managed service, or professional security, legal, or financial advice. Nothing here creates a consulting relationship or a guarantee of any outcome, including employment, promotion, or income.

Intentionally insecure code

Parts of this lab are deliberately insecure for teaching, so you can see a weakness, understand it, and harden it. Never deploy the weak configurations to production, never expose them to the public internet, and never use them to hold real data. Deploy only in an isolated sandbox cloud account you control.

Authorized use only

You may only deploy, scan, attack, or test infrastructure that you own or are explicitly authorized in writing to test. Running security tooling against systems you do not own may be illegal. You are solely responsible for complying with all applicable laws and with the terms of your cloud provider, including the AWS Customer Agreement and Acceptable Use Policy.

You own the costs and the risk

You are responsible for every charge incurred in your cloud account, including any resource you forget to destroy. Always run the teardown step when you finish. Vigilantia Technologies INC is not responsible for any cloud charges, data loss, service interruption, security incident, or other damage arising from your use of these Materials.

Provided as is, no warranty

The Materials are provided as is, without warranty of any kind, express or implied. To the maximum extent permitted by law, Vigilantia Technologies INC and Zach Marcy are not liable for any direct, indirect, incidental, consequential, or special damages arising from the use of or inability to use the Materials.

Your responsibility

You are an adult professional making your own decisions. Verify what you deploy, understand what you run, and secure what you build. The methods here reflect one practitioner's approach and are a starting point for your own judgment, not a substitute for it.

© 2026 Vigilantia Technologies INC. “HackWithZach” and the HackWithZach logo are trademarks of Vigilantia Technologies INC. All rights reserved. Questions: zach@hackwithzach.com